SIMATS ENGINEERING

Scenario-Based Assessment

COURSE NAME:Software Testing **COURSE CODE: CSA3714**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Mathusri P-192311363(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

Question 1: Online Banking Fund Transfer Testing

1. Problem understanding

An online banking system frequently fails during fund transfers, especially at minimum and maximum transfer limits. This indicates that the existing test cases may not adequately test boundary values and invalid inputs. Random testing alone may miss important conditions.

2. Application of black-box testing techniques

Black-box testing checks whether the system gives the correct output for an input without examining the internal code.

- Equivalence Class Partitioning (ECP): Divide inputs into valid and invalid groups.
- Boundary Value Analysis (BVA): Test values immediately below, at, and above the limits.

Assume the permitted transfer amount is ₹100 to ₹50,000. These are illustrative limits; actual limits must follow the application requirements.

Test case	Transfer amount	Expected result
TC1	₹99	Reject
TC2	₹100	Accept
TC3	₹101	Accept
TC4	₹49,999	Accept
TC5	₹50,000	Accept
TC6	₹50,001	Reject
TC7	₹0	Reject
TC8	Negative amount	Reject
TC9	Amount greater than balance	Reject
TC10	Non-numeric input	Reject

3. Analysis of random testing

Random testing selects inputs randomly. It can discover unexpected defects, but it does not guarantee that minimum and maximum boundaries are tested. Repeated random inputs may also waste testing time.

Therefore, random testing should be retained as a supplementary technique, not used as the primary strategy.

4. Improved testing strategy

1. Identify all transfer rules and limits.
2. Apply ECP to valid and invalid inputs.
3. Apply BVA to minimum and maximum values.
4. Test insufficient balance, invalid account numbers, and daily transfer limits.
5. Test network failure and duplicate submissions.

6. Verify that failed transfers do not incorrectly deduct money.
7. Automate regression tests and repeat them after code changes.

5. Justification and conclusion

ECP reduces unnecessary test cases by grouping similar inputs, while BVA detects errors near critical limits. Additional negative and failure-recovery tests improve reliability. Combining these methods with supplementary random testing provides better defect detection than relying on random inputs alone.

Question 2: Healthcare Application – White-Box Testing

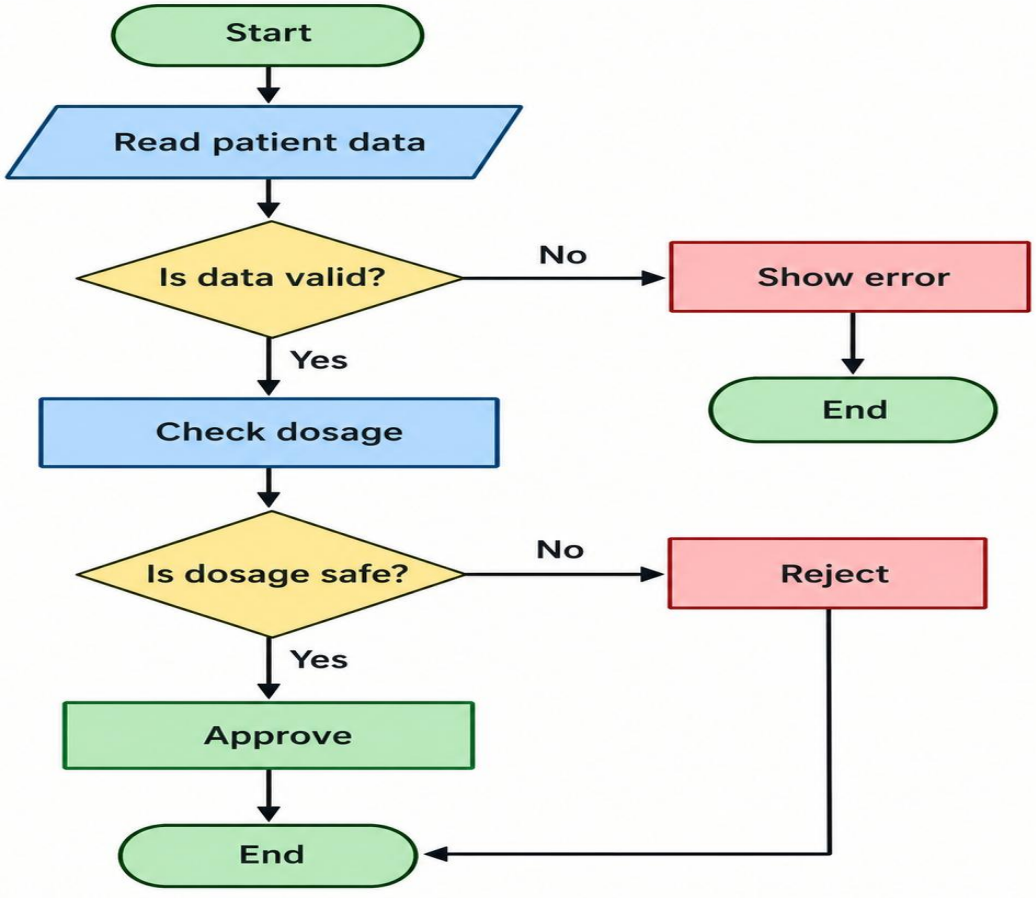
1. Problem understanding

A healthcare application released with critical defects may have insufficient code coverage, missing execution paths, or untested conditions. Because healthcare software can affect patient safety, testing must verify both normal and exceptional situations.

2. Application of white-box testing

White-box testing examines the internal code structure, decisions, branches, and execution paths.

Consider an illustrative patient dosage validation module:



The control flow graph (CFG) contains nodes representing operations and decisions, and edges representing possible control flow.

3. Example test cases

Test case	Input condition	Expected result
TC1	Valid patient data and safe dosage	Approve
TC2	Invalid patient data	Show validation error
TC3	Valid data but unsafe dosage	Reject
TC4	Missing dosage value	Show validation error
TC5	Dosage at the permitted upper limit	Apply specified limit rule
TC6	Invalid or unavailable patient record	Handle safely and report error

The dosage limits and exact approval rules must be obtained from the healthcare requirements.

4. Evaluation of coverage criteria

- Statement coverage: Checks whether every executable statement runs at least once.
- Branch coverage: Checks whether every decision outcome, such as true and false, executes.
- Path coverage: Checks execution sequences through the program.

Statement coverage alone is insufficient because a test may execute every statement without testing every decision outcome. Complete path coverage may be impractical when loops create many or infinitely many possible paths.

5. Improved testing strategy

1. Construct the CFG from the actual source code.
2. Identify all decision outcomes and important execution paths.
3. Design tests for valid, invalid, boundary, and exceptional inputs.
4. Measure statement and branch coverage.
5. Use basis-path testing to cover independent paths.
6. Test missing records, invalid data, and failure handling.
7. Perform integration, regression, and safety-focused testing.

6. Justification and conclusion

CFG-based testing identifies untested branches and execution paths. Combining branch coverage, basis-path testing, and realistic input tests improves defect detection. For safety-critical healthcare software, coverage measurements must be supported by requirement-based tests and risk analysis; no single coverage percentage proves that the application is completely safe.

Question 3: Retail E-Commerce – Price, Quantity and Discount Testing

1. Problem understanding

The e-commerce system accepts price, quantity, and discount values. Errors occur when these inputs are used in particular combinations. The current testing may check each input separately but fail to verify their combined effect on the final amount.

2. Application of black-box testing techniques

Equivalence Class Partitioning (ECP) divides inputs into valid and invalid groups.

Input	Valid class	Invalid class
Price	Price greater than ₹0	Zero, negative, or non-numeric
Quantity	Positive whole number	Zero, negative, or decimal quantity if not allowed
Discount	Within the permitted discount range	Negative or excessive discount

Boundary Value Analysis (BVA) tests values near the limits. Assume, for illustration, that the maximum discount is 50%.

Test case	Price	Quantity	Discount	Expected result
TC1	₹100	1	0%	₹100
TC2	₹100	1	1%	₹99
TC3	₹100	1	49%	₹51
TC4	₹100	1	50%	₹50
TC5	₹100	1	51%	Reject
TC6	₹0	1	10%	Reject
TC7	₹100	0	10%	Reject
TC8	₹100	2	10%	₹180

3. Decision table testing

Decision table testing verifies combinations of conditions and their corresponding actions.

Conditions/actions	Rule 1	Rule 2	Rule 3	Rule 4
Valid price	Yes	Yes	Yes	No
Valid quantity	Yes	Yes	No	Yes
Valid discount	Yes	No	Yes	Yes
Expected action	Calculate total	Reject discount	Reject quantity	Reject price

This identifies errors that may occur only when two or more conditions interact.

4. Analysis and improved approach

Testing each field independently cannot guarantee correct calculation of the final total. The revised approach should:

1. Apply ECP to all input fields.
2. Test minimum and maximum boundaries.
3. Use decision tables for combinations of price, quantity, and discount.
4. Verify the formula: Total = Price × Quantity × (1 − Discount/100).
5. Check rounding, currency formatting, and zero-value totals.
6. Use pairwise testing for larger combinations, while retaining tests for high-risk combinations.
7. Automate regression tests for pricing rules.

5. Justification and conclusion

ECP reduces redundant tests, BVA detects boundary errors, and decision tables identify combination-related defects. Pairwise testing can reduce the number of tests when many inputs exist, but it should not replace exhaustive testing of critical business rules. Together, these techniques improve test completeness and pricing accuracy.

Question 4: Complex Nested Conditions and Loops – Path Testing

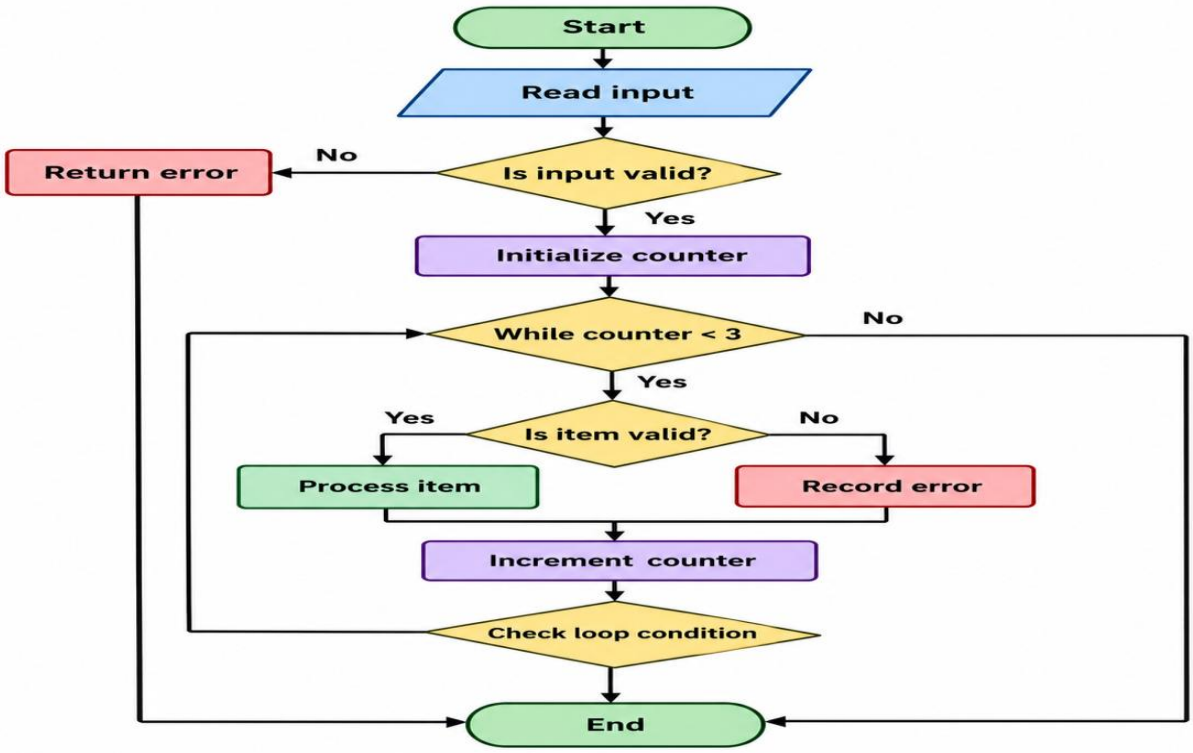
1. Problem understanding

A software module contains nested conditions and loops, but only partial path coverage has been achieved. This means some execution sequences may remain untested. Defects can occur when particular combinations of decisions or loop iterations are executed.

2. Application of white-box testing

White-box testing examines the internal logic of the program. Path testing uses a Control Flow Graph (CFG) to identify execution paths.

Consider the following illustrative pseudocode:



3. Independent paths

An independent path introduces at least one new edge that has not appeared in the previously selected paths.

Example paths:

- P1: Invalid input → Return error → End.
- P2: Valid input → Loop executes → Item valid → Process → Exit loop.
- P3: Valid input → Loop executes → Item invalid → Record error → Exit loop.
- P4: Valid input → Loop executes repeatedly until the limit is reached.

The paths must be verified against the actual program's CFG.

4. Cyclomatic complexity

Cyclomatic complexity estimates the number of linearly independent paths.

$$V(G)=E-N+2P$$

Where:

- E = number of edges.
- N = number of nodes.
- P = number of connected components; for one connected CFG, $P=1$.

For a single connected graph with three decision nodes, the complexity is normally:

$$V(G)=3+1=4$$

This indicates four independent basis paths, subject to the actual graph structure.

5. Improved testing strategy

1. Draw the CFG from the source code.
2. Identify decision nodes, loop entries, and exits.
3. Calculate cyclomatic complexity.
4. Create tests for each independent path.
5. Test zero, one, typical, and maximum loop iterations.
6. Test invalid inputs and early exits.
7. Verify loop termination and boundary conditions.
8. Measure branch coverage and execute regression tests.

6. Justification and conclusion

Partial path coverage may leave combinations of nested conditions untested. CFG construction and basis-path testing systematically identify independent paths. Loop-boundary tests also detect incorrect termination and repeated execution. Since loops can create very large numbers of paths, the practical goal is to test independent paths and all critical loop conditions rather than claim unrestricted complete path coverage.

Question 5: Financial Application – Hybrid Black-Box and White-Box Testing

1. Problem understanding

A financial application uses both black-box and white-box testing, but defects still appear in production. This suggests that using two testing methods does not automatically guarantee sufficient coverage. The team may have missed boundary conditions, internal execution paths, integration defects, or failure scenarios.

2. Application of black-box techniques

Black-box testing validates the application's behaviour against requirements.

- ECP: Divide transaction amounts and account inputs into valid and invalid classes.
- BVA: Test minimum and maximum transaction limits.
- Decision table testing: Test combinations such as account status, balance, transaction type, and daily limit.

Example: If a transaction is permitted between ₹100 and ₹50,000, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001.

3. Application of white-box techniques

White-box testing checks the internal program logic.

- CFG: Maps statements, decisions, and control flow.
- Branch coverage: Tests every decision outcome.
- Basis-path testing: Executes independent paths.
- Code inspection: Identifies incorrect calculations, missing validations, and unsafe error handling.

4. Proposed hybrid test cases

Test case	Testing technique	Expected result
TC1	ECP	Valid transaction succeeds
TC2	BVA	Minimum permitted amount is handled correctly
TC3	BVA	Amount above maximum is rejected
TC4	Negative testing	Insufficient balance is rejected
TC5	Branch testing	Both success and failure branches execute
TC6	Path testing	All identified critical transaction paths execute
TC7	Failure recovery	Network failure does not cause incorrect deductions
TC8	Regression testing	Previously fixed transaction defects do not recur

5. Analysis and improved approach

The testing team should:

1. Map every financial requirement to one or more test cases.
2. Apply ECP, BVA, and decision tables.
3. Construct CFGs for critical transaction logic.
4. Measure statement and branch coverage.
5. Test database transactions, concurrent requests, and rollback behaviour.
6. Use automation for repeated regression tests.
7. Prioritize high-risk financial operations.
8. Review production incidents and add regression tests for each confirmed defect.

6. Justification and conclusion

Black-box testing detects incorrect external behaviour, while white-box testing identifies missing internal paths and logic errors. Their combination provides broader coverage than either approach alone. Requirement traceability, risk-based testing, and regression testing help prevent recurring production defects and improve financial accuracy.

Question 6: HR Payroll – Bonus Calculation at 5, 10 and 15 Years

1. Problem understanding

The payroll system calculates employee bonuses using years of service and performance ratings. Incorrect payouts occur at exactly 5, 10, and 15 years. These failures strongly suggest errors in threshold comparisons, such as using the wrong greater-than or greater-than-or-equal-to operator.

2. Application of Boundary Value Analysis

Assume the bonus rules change at 5, 10, and 15 years. The exact bonus amounts and eligibility rules must be taken from the payroll specification.

Test case	Years of service	Purpose
TC1	4.99	Just below 5 years
TC2	5.00	Exactly 5 years
TC3	5.01	Just above 5 years
TC4	9.99	Just below 10 years
TC5	10.00	Exactly 10 years
TC6	10.01	Just above 10 years
TC7	14.99	Just below 15 years
TC8	15.00	Exactly 15 years
TC9	15.01	Just above 15 years

If service is recorded only in completed years, use 4, 5, 6 and equivalent integer values around each threshold instead.

3. Application of Equivalence Class Partitioning

Divide service periods into groups that follow the same bonus rule. For example, the classes may be less than 5 years, 5 to less than 10 years, 10 to less than 15 years, and 15 years or more. These classes are illustrative; the official rules determine the correct groups.

Performance ratings should also be partitioned into valid ratings and invalid or missing ratings.

4. Combined test cases

Test case	Service	Performance rating	Expected result
TC1	5 years	Valid high rating	Correct 5-year bonus
TC2	10 years	Valid medium rating	Correct 10-year bonus
TC3	15 years	Valid high rating	Correct 15-year bonus
TC4	5 years	Invalid rating	Validation error
TC5	4 years	Valid rating	Bonus for the applicable lower service class
TC6	16 years	Valid rating	Bonus for the applicable upper service class

5. Analysis and justification

Exploratory testing is useful for discovering unexpected behaviour, but it depends heavily on the tester's experience and may overlook exact thresholds. BVA systematically checks values around each known boundary. ECP reduces redundant testing by grouping inputs with equivalent expected behaviour.

The revised approach should also compare calculated payouts with an independently verified expected result and test rounding, missing employee data, and payroll regression.

6. Conclusion

BVA is particularly suitable because the reported defects occur at exact service thresholds. ECP provides broader coverage of valid and invalid employee categories. Combining both methods with automated calculation checks reduces payout errors and improves payroll reliability.

Question 7: Laboratory Temperature Control – 100% MC/DC Testing

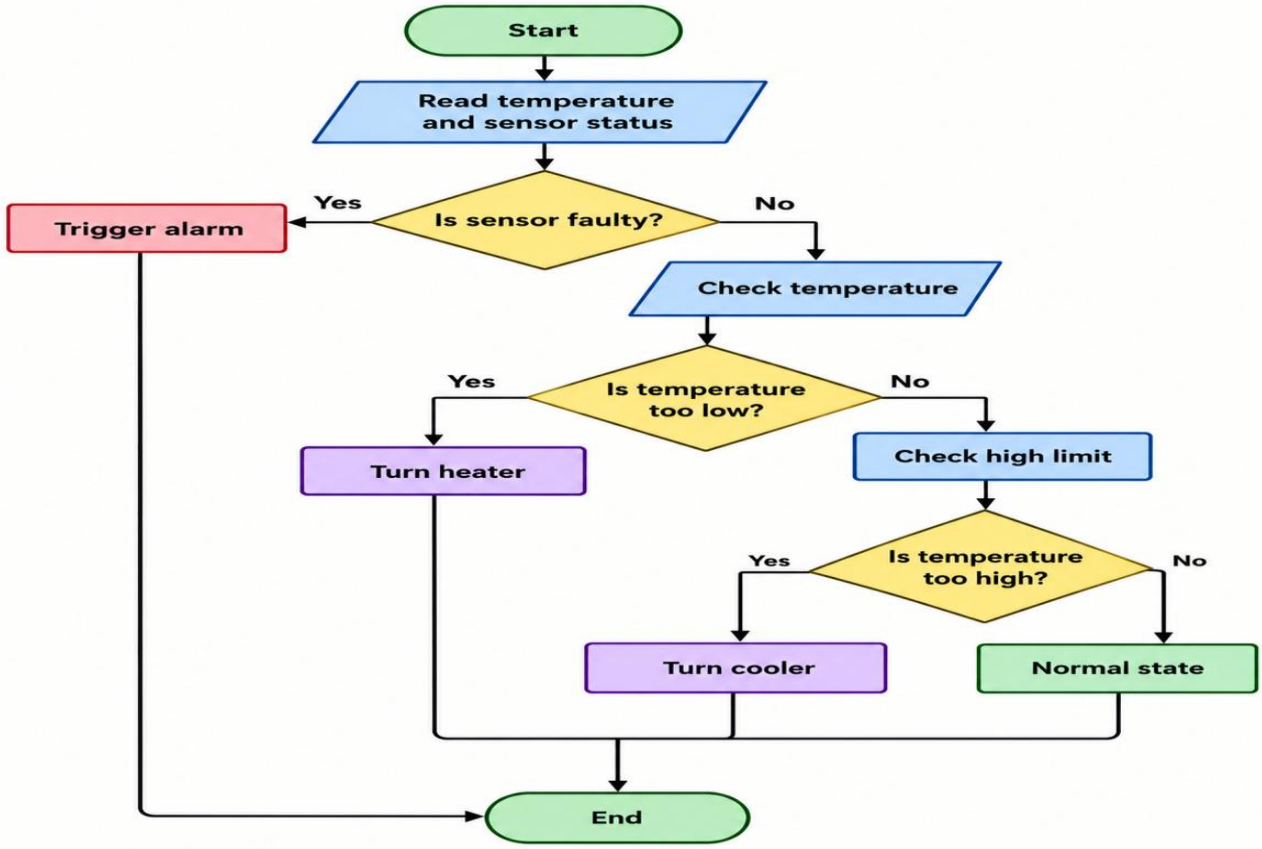
1. Problem understanding

The laboratory temperature control system uses if-else conditions to control heaters, coolers, and alarms. An alarm failed to activate during a sensor malfunction. This suggests that one or more combinations of sensor conditions were not adequately tested.

Because the system is safety-critical, testing must verify that every important condition can independently affect the final decision.

2. Control Flow Graph (CFG)

Consider this illustrative logic:



The actual CFG must reflect the source code and the required alarm behaviour.

3. Branch coverage analysis

Branch coverage checks whether every decision outcome has been executed.

For example:

- Sensor faulty: true and false.
- Temperature too low: true and false.
- Temperature too high: true and false.

Testing only normal temperatures would leave the fault and alarm branches untested.

4. Modified Condition/Decision Coverage (MC/DC)

MC/DC requires every individual Boolean condition to be shown to independently affect the outcome of the overall decision.

Consider the illustrative alarm decision:

Alarm = SensorFault OR OverTemperature

Test	SensorFault	OverTemperature	Alarm
TC1	False	False	False
TC2	True	False	True
TC3	False	True	True
TC4	True	True	True

- Comparing TC1 and TC2 shows the independent effect of SensorFault.
- Comparing TC1 and TC3 shows the independent effect of OverTemperature.

Thus, the two conditions independently affect the decision outcome.

5. Improved testing strategy

1. Identify all Boolean conditions in the control logic.
2. Construct the CFG.
3. Create test cases for every branch.
4. Design MC/DC pairs for each condition.
5. Simulate sensor failures, disconnected sensors, and abnormal readings.
6. Verify heater, cooler, and alarm responses.
7. Test recovery after the sensor returns to normal.
8. Record coverage evidence and repeat regression tests after changes.

6. Justification and conclusion

Statement coverage alone cannot demonstrate that individual conditions independently influence a decision. MC/DC gives stronger evidence of decision-logic correctness. For a safety-critical temperature controller, combining MC/DC, branch testing, fault injection, and hardware integration testing improves confidence that alarms operate correctly under abnormal conditions.

Question 8: Travel Booking Portal – Promo Code Combinations

1. Problem understanding

A travel booking portal allows multiple promo codes, but some combinations produce a negative total amount. The problem may arise because discounts are applied in the wrong order, incompatible promotions are combined, or the system fails to enforce a minimum payable amount.

Testing price, promo type, and user tier separately cannot reveal every interaction between them.

2. Decision Table Testing

Assume the portal uses the following illustrative rules:

- The booking price must be positive.
- Only compatible promotions can be applied.
- The user must meet the promotion's eligibility conditions.
- The final payable amount cannot be negative.

Condition/action	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
Valid booking price	Yes	Yes	Yes	No
Eligible user tier	Yes	Yes	No	Any
Compatible promo codes	Yes	No	Any	Any
Expected action	Calculate total	Reject combination	Reject promotion	Reject booking
Negative total allowed	No	No	No	No

3. Example test cases

Test case	**Input combination**	**Expected result**
TC1	Positive price, eligible tier, one valid promo	Apply discount correctly
TC2	Positive price, eligible tier, compatible promos	Apply permitted combination
TC3	Positive price, incompatible promos	Reject or apply specified priority rule
TC4	Positive price, ineligible user tier	Reject promotion
TC5	Price of zero	Reject booking or apply specified rule
TC6	Discounts exceed booking price	Prevent negative payable amount
TC7	Invalid or expired promo code	Display appropriate error

4. Analysis and optimized approach

Cause-effect graphing can also be used to map relationships between input conditions and outputs. For example, a valid price, eligible tier, and compatible promotion should lead to a discounted total. An invalid price or incompatible promotion should lead to rejection.

The testing team should:

1. Identify valid and invalid input classes.
2. Create a decision table from business requirements.
3. Test each distinct rule and important rule interaction.
4. Verify discount order and rounding.
5. Test multiple-code limits and expired codes.
6. Apply pairwise testing to larger input combinations.
7. Check that the final total is never negative.
8. Automate regression tests for promotion rules.

5. Justification and conclusion

Decision table testing is effective because it tests combinations of conditions and their corresponding actions. It detects defects that isolated input tests may miss. Selecting tests for each distinct rule and high-risk interaction maximizes defect detection while reducing unnecessary duplication.

Question 9: Database Migration Tool – Infinite Loop Crashes

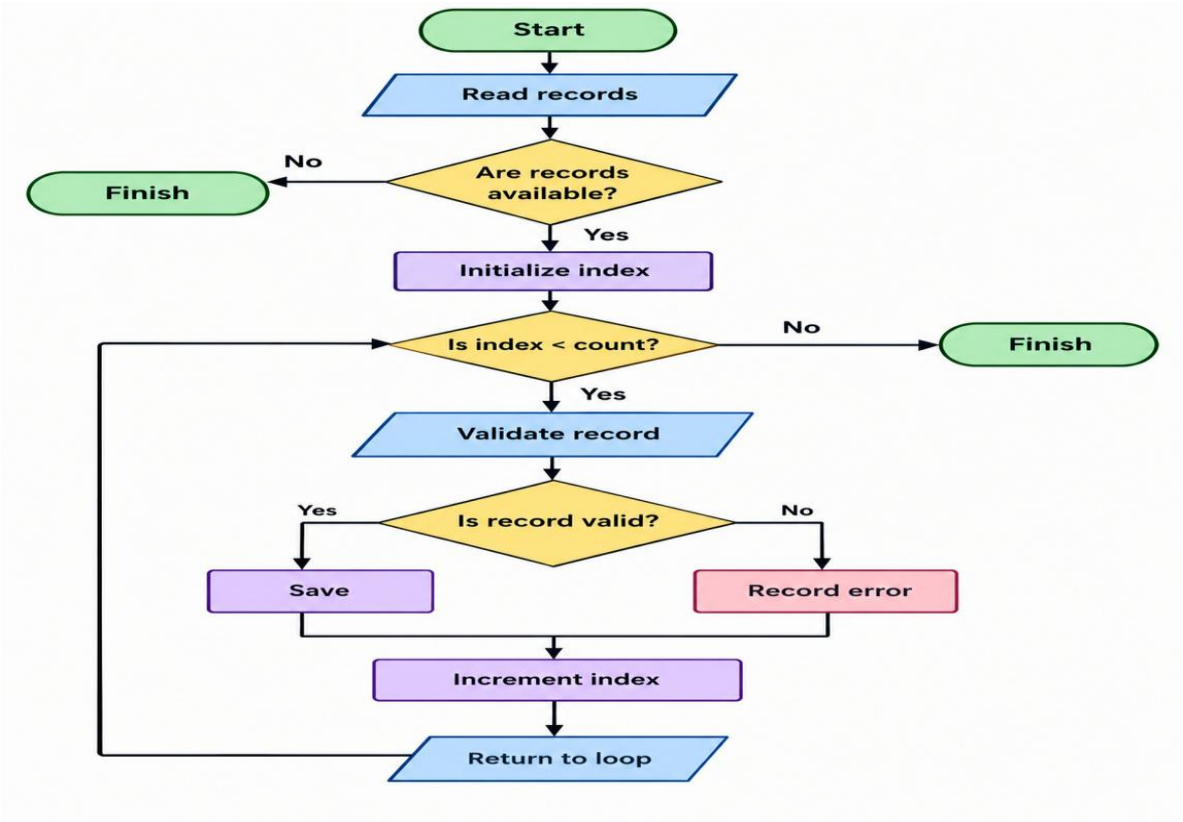
1. Problem understanding

The database migration tool reports high statement coverage, but infinite-loop crashes occur in production. This means the tests execute many statements but do not adequately verify loop termination, decision outcomes, or repeated execution behaviour.

Statement coverage measures executed statements; it does not prove that loops terminate for all relevant inputs.

2. Application of white-box testing

Consider the following illustrative pseudocode:



If the index is not incremented on every required path, or if the loop condition never becomes false, the program may run indefinitely.

3. Why statement coverage is insufficient

A test may execute every statement once without testing repeated loop iterations. It may also fail to detect:

- Incorrect loop termination conditions.
- Missing counter updates.
- Incorrect handling of empty datasets.
- Repeated processing of the same record.
- Exceptional paths that skip loop updates.

4. Cyclomatic complexity

Cyclomatic complexity estimates the number of independent paths:

\[V(G)=E-N+2P \]

For a connected graph, it can also be calculated as the number of decision nodes plus one.

For example, if a simplified migration routine has three decision nodes:

\[V(G)=3+1=4 \]

This indicates four independent basis paths for that simplified graph. The actual value must be calculated from the implemented CFG.

5. Proposed test cases

Test case	**Input**	**Expected result**
TC1	Empty dataset	Finish without looping
TC2	One valid record	Process one record and terminate
TC3	Multiple valid records	Process all records
TC4	Invalid record	Record error and continue or stop as specified
TC5	Maximum permitted dataset	Complete within acceptable time
TC6	Repeated or malformed record	Handle safely without infinite looping
TC7	Database connection failure	Exit or retry according to the retry limit

6. Improved testing strategy

1. Construct the CFG.
2. Calculate cyclomatic complexity.
3. Test each independent path.
4. Test zero, one, and multiple loop iterations.
5. Verify that every loop iteration makes progress or exits.
6. Introduce malformed records and simulated database failures.
7. Use timeouts and watchdogs to detect non-terminating execution.
8. Measure coverage and perform regression testing.

7. Justification and conclusion

Statement coverage alone does not test loop termination. Basis-path testing, loop-boundary testing, and fault-injection testing reveal missing updates and incorrect conditions. Cyclomatic complexity helps identify the minimum basis-path set for a given CFG. These techniques improve migration reliability and reduce the risk of production crashes.

Question 10: Food Delivery App – State Transition Testing

1. Problem understanding

The food delivery app has four main order states: Ordered, Preparing, Out for Delivery, and Delivered. The app becomes stuck when the driver cancels during delivery. This indicates that the state-transition design may not properly handle cancellation and recovery.

ECP and BVA test input categories and numeric boundaries, but they do not fully represent the sequence of events in a dynamic workflow.

2. Application of State Transition Testing

State Transition Testing verifies the system's response when it moves from one state to another because of an event.

The following is an illustrative state model:

Ordered

Preparing

Out for Delivery

Normal delivery

Delivered

Driver cancels during delivery

Cancellation / Reassignment

The cancellation state and its next action must follow the application's actual requirements. A reassignment, cancellation, or refund may be appropriate depending on the order's status.

3. Example test cases

Test case	**Initial state and event**	**Expected result**
TC1	Ordered → restaurant accepts	Preparing
TC2	Preparing → order ready	Out for Delivery after pickup
TC3	Out for Delivery → delivery completed	Delivered
TC4	Out for Delivery → driver cancels	Cancellation/reassignment handled correctly
TC5	Delivered → cancellation request	Reject an invalid transition or follow the specified policy
TC6	Cancellation during network failure	Recover without inconsistent order status
TC7	Repeated cancellation event	Do not duplicate refunds or reassignment actions

4. Analysis and improved approach

The testing team should:

1. List every valid state and event.
2. Define permitted and prohibited transitions.
3. Test each important transition.
4. Test cancellation from every eligible state.
5. Test repeated events, network interruptions, and delayed notifications.
6. Verify consistency between the app, restaurant, driver, and payment system.
7. Automate transition and regression tests.

5. Justification and conclusion

State Transition Testing detects errors caused by event order and invalid state changes. It is more suitable than ECP and BVA alone for workflows whose behaviour depends on the previous state. A clearly defined cancellation path prevents stuck orders, inconsistent status displays, and incorrect refunds.

Question 11: Cybersecurity Authentication – Hybrid Testing

1. Problem understanding

The authentication module checks password strength, OTP timing, and IP location, but brute-force vulnerabilities were found. This indicates that testing may have focused on normal login attempts without adequately testing repeated failures, security controls, and internal authentication paths.

2. Application of black-box and white-box techniques

- Black-box testing: Tests login behaviour using valid and invalid credentials without examining internal code.

- Error Guessing: Uses experience to predict likely defects, such as missing rate limits or reusable OTPs.
- Path Analysis: Examines the internal decision paths for password validation, OTP verification, and IP checks.

3. Example test cases

Test case	Scenario	Expected result
TC1	Correct password and valid OTP	Allow login if other requirements pass
TC2	Incorrect password	Reject login
TC3	Incorrect OTP	Reject login
TC4	Expired OTP	Reject login
TC5	Repeated failed login attempts	Apply rate limiting or specified protective controls
TC6	OTP reused after successful login	Reject reused OTP
TC7	Unusual IP location	Apply the required risk-based verification policy
TC8	Authentication service failure	Fail safely without bypassing authentication

4. Hybrid testing strategy

1. Apply ECP to valid and invalid passwords, OTPs, and IP conditions.
2. Use BVA to test OTP expiration times and attempt limits.
3. Use error guessing to identify missing rate limits, OTP reuse, and account lockout weaknesses.
4. Construct a CFG for the authentication logic.
5. Identify independent paths through password, OTP, and IP verification.
6. Test both successful and unsuccessful authentication branches.
7. Perform authorized security testing for brute-force resistance.
8. Verify audit logging, session handling, and regression after fixes.

5. Analysis and justification

Black-box tests identify observable security failures, while path analysis helps uncover missing internal checks. Error guessing adds practical threat scenarios that formal test cases may overlook. Rate limiting, appropriate delays, monitoring, and carefully designed account protection help reduce brute-force risk.

6. Conclusion

A hybrid strategy provides stronger security coverage than testing successful logins alone. Combining negative testing, error guessing, path analysis, and authorized security testing helps identify authentication weaknesses and improve resistance to repeated login attacks.

Question 12: E-Learning Platform – Subscription Expiration and Upgrade Testing

1. Problem understanding

The e-learning platform provides Free, Basic, and Premium access. Errors occur when a user upgrades exactly on the subscription expiration date. This suggests inadequate testing of date boundaries, subscription eligibility, or the order in which expiration and upgrade rules are applied.

2. Application of Equivalence Class Partitioning

Assume the subscription expires at a specified date and time. The test cases should distinguish between:

- Before the expiration instant.
- Exactly at the expiration instant.
- After the expiration instant.

The application's documented time zone and expiration policy must determine the expected result.

3. Date-boundary test cases

Test case	Upgrade request time	Purpose
TC1	Just before expiration	Verify eligible upgrade
TC2	Exactly at expiration	Verify the documented boundary rule
TC3	Just after expiration	Verify expired-subscription handling
TC4	Well before expiration	Verify normal upgrade
TC5	Long after expiration	Verify expired account handling
TC6	Invalid expiration date	Reject or safely handle invalid data

4. Subscription equivalence classes

Subscription type	Input condition	Expected result
Free	Valid account	Free access
Basic	Active subscription	Basic access
Premium	Active subscription	Premium access
Basic	Expired subscription	Apply expiration policy
Premium	Invalid subscription dates	Reject or flag inconsistent data
Any tier	Unauthorized upgrade request	Reject request

5. Improved testing strategy

1. Define the rules for subscription activation, expiration, and upgrades.
2. Partition dates into valid and invalid classes.
3. Test just before, exactly at, and just after expiration.
4. Test all subscription tiers and important upgrade combinations.
5. Verify payment success and failure.
6. Test time-zone differences and date changes at midnight.

7. Ensure access rights update correctly after an upgrade.
8. Automate date-based regression tests.

6. Analysis and justification

Testing only ordinary dates misses defects at the exact expiration boundary. ECP provides representative tests for date ranges and subscription types, while BVA concentrates on the transition point. Combining them helps identify incorrect comparisons, outdated permissions, and inconsistent access rights.

7. Conclusion

A structured date-based test suite ensures predictable subscription transitions. Testing exact expiration times, tier changes, and payment outcomes improves reliability and prevents users from receiving incorrect access.

Question 13: Smart-Home Lighting Controller – Branch and Condition Coverage

1. Problem understanding

The smart-home lighting controller uses switch-case statements and complex Boolean expressions to adjust brightness based on ambient light sensors. Relying only on statement coverage may leave important branches and sensor combinations untested, leading to incorrect lighting behaviour.

2. Application of white-box testing

White-box testing examines the internal control logic.

- Statement coverage: Executes each statement at least once.
- Branch coverage: Exercises each decision outcome.
- Condition coverage: Makes each individual Boolean condition evaluate to both true and false.
- Switch-case testing: Tests every relevant case and the default case.

3. Illustrative decision logic

Assume the controller turns on a light when the room is dark and a person is detected.

LightOn = Dark AND PersonDetected

Test case	Dark	Person detected	Expected light state
TC1	False	False	Off
TC2	False	True	Off
TC3	True	False	Off
TC4	True	True	On

These four tests exercise both outcomes of each condition and both outcomes of the overall decision.

4. Switch-case test cases

Assume the controller has three illustrative brightness levels.

Test case	Ambient light category	Expected result
TC1	Low ambient light	High brightness
TC2	Medium ambient light	Medium brightness
TC3	High ambient light	Low brightness
TC4	Invalid sensor value	Safe fallback or error handling
TC5	Sensor disconnected	Defined fault response

The exact categories and brightness values must be derived from the product requirements.

5. Improved white-box strategy

1. Draw a CFG of the control algorithm.
2. Identify all Boolean conditions and switch cases.
3. Test every branch and individual condition.
4. Test boundary sensor readings.
5. Simulate disconnected, noisy, and incorrect sensors.
6. Check the controller's response to rapidly changing readings.
7. Perform hardware-software integration testing.
8. Measure coverage and add regression tests for every defect.

6. Analysis and justification

Statement coverage may execute the code for turning lights on and off without testing all conditions that control those actions. Branch and condition coverage provide stronger evidence that the decision logic has been exercised. Sensor fault tests and hardware integration tests detect problems that code coverage alone cannot reveal.

7. Conclusion

A combination of branch coverage, condition coverage, switch-case testing, and hardware integration testing improves the reliability of the lighting controller. It helps prevent incorrect brightness changes and ensures that sensor failures are handled safely.

Question 14: Logistics Management – Robust Boundary Value Analysis

1. Problem understanding

The logistics system calculates shipping costs using package weight, destination zone, and delivery speed. Testers failed to detect problems with zero and negative weights. This indicates that the current validation tests did not adequately include invalid boundary values.

2. Application of Robust Boundary Value Analysis

Robust BVA tests values below, at, and above valid boundaries, including invalid values.

Assume, for illustration, that the permitted package weight is greater than 0 kg and up to 100 kg inclusive. The actual limits must follow the shipping requirements.

Test case	Weight	Expected result
TC1	−0.01 kg	Reject
TC2	0 kg	Reject
TC3	0.01 kg	Accept
TC4	1 kg	Calculate shipping cost
TC5	99.99 kg	Accept
TC6	100 kg	Accept
TC7	100.01 kg	Reject
TC8	Blank weight	Show validation error
TC9	Non-numeric weight	Show validation error

3. Testing additional input combinations

Shipping cost depends on more than weight, so additional cases are necessary.

Test case	Weight	Destination	Delivery speed	Expected result
TC10	Valid	Local zone	Standard	Correct standard charge
TC11	Valid	Remote zone	Standard	Correct remote-zone charge
TC12	Valid	Local zone	Express	Correct express charge
TC13	Invalid	Any zone	Any speed	Reject weight
TC14	Valid	Invalid zone	Standard	Reject invalid destination
TC15	Valid	Local zone	Invalid option	Reject invalid delivery speed

4. Improved testing strategy

1. Identify minimum and maximum permitted weights.
2. Test invalid values below the minimum and above the maximum.
3. Apply ECP to valid and invalid weights, destinations, and delivery options.
4. Test combinations that affect shipping charges.
5. Verify currency rounding and charge calculations.
6. Test missing, malformed, and extremely large values.
7. Confirm that invalid inputs cannot produce negative or misleading charges.
8. Automate validation and regression tests.

5. Analysis and justification

Testing only typical positive weights does not establish that the system correctly rejects zero or negative weights. Robust BVA systematically tests these invalid boundaries. ECP and combination testing extend coverage to destination and delivery-speed rules.

Negative scenarios are essential because invalid inputs can cause incorrect billing, calculation errors, and inconsistent shipment records.

6. Conclusion

Robust BVA detects defects near the limits of permitted weight, while ECP covers representative valid and invalid inputs. Together with combination and regression tests, these techniques improve input validation, shipping-cost accuracy, and system robustness.

Question 15: Medical Imaging – Basis Path Testing and Prime Data Sizes

1. Problem understanding

Medical imaging software processes large datasets using parallel-processing loops. Inconsistent results occur only when the dataset size is a prime number. The team's reliance on random testing is insufficient because randomly selected sizes may not consistently include prime numbers or expose particular loop and parallel-processing behaviours.

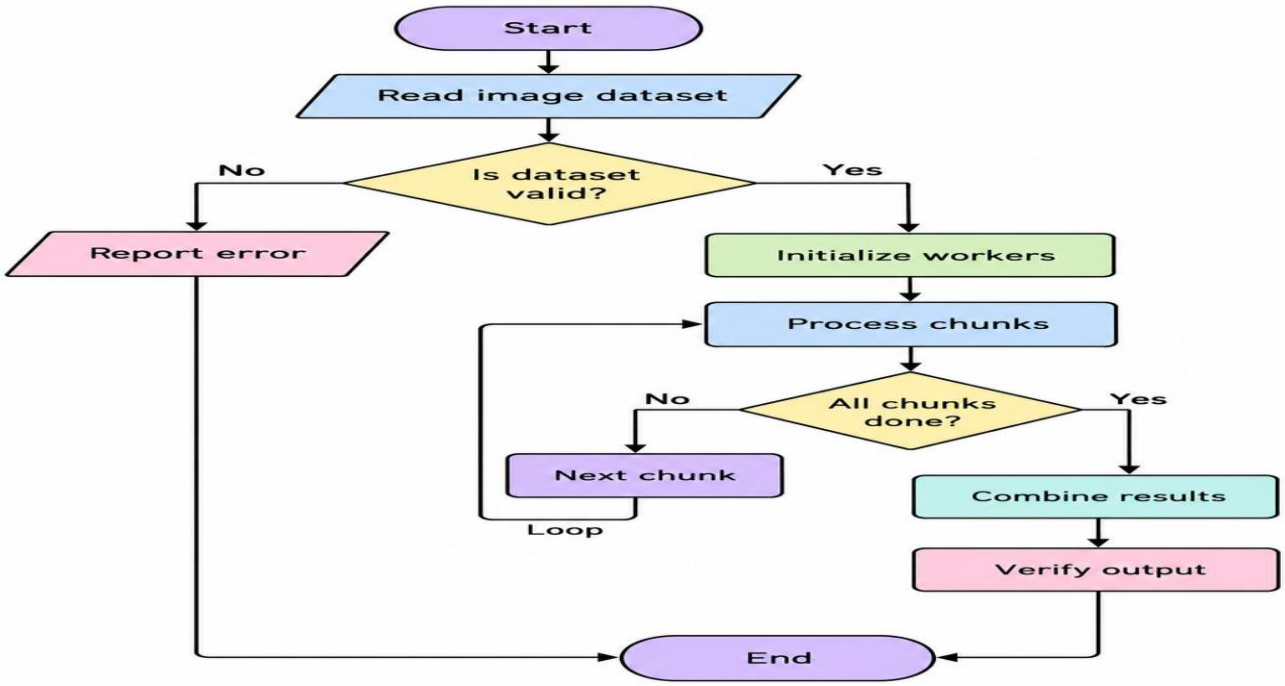
Since medical imaging results can influence clinical decisions, correctness and repeatability are important.

2. Application of white-box testing

White-box testing examines the internal code, decisions, loops, and execution paths.

Basis Path Testing uses a CFG and cyclomatic complexity to identify a set of linearly independent paths. Each selected path should be executed at least once.

Consider this illustrative pseudocode:



This is a conceptual CFG; the actual graph must be derived from the implementation.

3. Cyclomatic complexity

The number of independent basis paths can be estimated using:

\[V(G)=E-N+2P \]

For a connected graph:

\[V(G)=\text{Number of decision nodes}+1 \]

For example, if the simplified processing module contains three decision nodes, its cyclomatic complexity is 4. This suggests four independent basis paths for that simplified CFG.

4. Example test cases

Test case	**Dataset size**	**Purpose**	**Expected result**
TC1	0	Empty dataset	Handle safely
TC2	1	Smallest positive dataset	Correct output
TC3	7	Small prime size	Correct output
TC4	8	Nearby composite size	Correct output
TC5	101	Larger prime size	Correct output
TC6	100	Nearby composite size	Correct output
TC7	Large permitted size	Stress and parallel processing	Correct output within limits
TC8	Size not divisible evenly among workers	Test uneven chunk allocation	No missing or duplicate data

TC9	Invalid or corrupted dataset	Error handling	Safe rejection or recovery

Prime and composite dataset sizes should be tested systematically, especially around the sizes that previously caused failures.

5. Improved testing strategy

1. Construct the CFG from the actual source code.
2. Calculate cyclomatic complexity.
3. Identify independent paths through validation, chunk processing, and result combination.
4. Test prime and composite dataset sizes.
5. Test zero, one, small, and maximum supported dataset sizes.
6. Test uneven division of work across parallel workers.
7. Compare parallel results against a trusted sequential implementation.
8. Repeat tests to detect inconsistent or non-deterministic results.
9. Measure code coverage and perform regression testing.

6. Analysis and justification

Random testing may miss the special data sizes that trigger the defect. Basis Path Testing provides systematic coverage of independent control-flow paths, while targeted prime-size tests address the reported failure pattern.

However, basis-path coverage alone cannot prove mathematical correctness or eliminate all parallel-processing defects. Comparing results with a trusted implementation, testing chunk boundaries, and repeating parallel executions provide additional evidence of correctness.

7. Conclusion

Combining Basis Path Testing, cyclomatic complexity, targeted prime-size tests, and parallel-processing validation improves defect detection. This structured strategy is more reliable than random testing alone and helps improve the mathematical accuracy, repeatability, and stability of medical imaging software.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification